NON-TECHNICAL & DEVELOPER HANDBOOK

How Tracking & Analytics Works in SpeakwithMaya

This document is written for **both non-technical team members** (founders, marketing, ad managers) and **developers**. For every tracking step in the student journey, you will find: (1) a plain-English explanation of what the student does on screen, (2) a real-life scenario, (3) what data is sent to Google Tag Manager / Meta Ads, and (4) the exact code file and line numbers.

1. Where Data Goes
Both **speakwithmaya.com** (landing page) and **app.speakwithmaya.com** (web app) send events into Google Tag Manager (`GTM-MVNHWL6X`) via `window.dataLayer`, plus include Facebook Domain Verification tags in `<head>`.

2. Automatic Privacy Protection
Before any event is sent out, our privacy filter automatically blocks **20 personal data fields** (phone numbers, student names, emails, OTP codes, credit card numbers). Private user data never leaks to ad networks.

3. Accurate, No-Duplicate Numbers
If a user refreshes their browser or their phone internet reconnects mid-call, the system remembers their event ID so registrations, purchases, and voice calls are **never counted twice**.

1. Complete Student Journey Funnel (Step 1 to Step 8)

Here is the full sequence of events from a visitor arriving on the landing page to purchasing a plan and practicing English with Maya:

Step	Tracking Event Name	What the Student Does (Plain English)	Where It Is Implemented (File & Lines)
Step 1 Website Setup	`gtm.js + facebook-domain-verification`	Visitor opens speakwithmaya.com or app.speakwithmaya.com; GTM & Facebook domain meta tags load in HTML <head>.	maya-landing/src/app/layout.tsx (L210–218) maya-frontend/src/app/+html.tsx (L17–28)
Step 2 Cookie Banner	`cookie_consent_update`	Visitor clicks 'Accept All' or 'Essential Only' on the landing page cookie banner.	maya-landing/src/components/CookieConsentBanner.tsx (L52–58)
Step 3 Account Sign Up	`sign_up`	New student enters their phone number and verifies the SMS OTP code for the very first time (ignores returning logins).	maya-frontend/src/utils/analytics.ts (L302–313) maya-frontend/src/app/onboarding/otp.tsx (L207–212)
Step 4 Finish Quiz	`onboarding_complete`	Student finishes all onboarding questions (goal, level, daily target) and their personalized plan is generated.	analytics.ts (L318–325) building-plan.tsx (L381) personalized-plan.tsx (L235, L642)

Step 5 View & Pick Plan	`view_plan_selection`
	`view_all_plans`
	`select_plan`
	`continue_starter`
		Student views the pricing screen, expands all plans, clicks a plan card (Basic/Pro/Premium/+30m/120m/300m), or continues on Starter.	analytics.ts (L164–237, L330–370)
			personalized-plan.tsx (L236, 643, 716, 819)
			upgrade.tsx (L381, 632, 967)
			common-popup.tsx (L415)

Step 6 Start Checkout	`begin_checkout`	Student clicks to pay via PayHere or confirms a 1-Click saved card upgrade.	analytics.ts (L356–365) personalized-plan.tsx (L537, 579) upgrade.tsx (L668, 703) common-popup.tsx (L432, 468)

Step 7 Payment Result	`purchase`
	`payment_failed`
	`payment_cancelled`
		Server verifies payment succeeded (purchase), card was declined (payment_failed), or student closed PayHere (payment_cancelled).	analytics.ts (L376–437)
			upgrade.tsx (L406–444, 672–743)
			personalized-plan.tsx (L261–299, 545–622)
			payhere.service.ts (L1916, 2889)

Step 8 AI Voice Call	`session_start` `session_complete`	Student connects to a live English speaking call with Maya and finishes the call (sends call duration in seconds).	analytics.ts (L442–460) maya-frontend/src/hooks/useLiveCall.ts (L656, L1490)

2. Detailed Scenarios & Exact Code Lines

[SCENARIO 1] Website <head> Setup: Google Tag Manager & Facebook Domain Verification

GTM-MVNHWL6X + meta tags

What Happens (Plain English): Every page on both speakwithmaya.com (Landing) and app.speakwithmaya.com (Web App) includes the Google Tag Manager script and the static Facebook Domain Verification <meta> tag directly inside the HTML <head> section.
Why It Matters for Business / Marketing: Facebook Business Manager requires the verification tag to be inside <head> in the raw HTML source (not injected late by JS). This proves ownership of both domains and initializes window.dataLayer immediately.

Real-Life Example: When Facebook's verification bot checks speakwithmaya.com or app.speakwithmaya.com, it immediately finds the matching <meta name='facebook-domain-verification'> tag inside <head> and marks both domains verified.

Exact Data Sent to GTM: `Landing: 4upgmm941w888kmdif1i6kjywtit3o | App: 78qfs00exvb1c95o4ctbms7pq1c0wc | GTM: GTM-MVNHWL6X`

Code File & Line Numbers: maya-landing/src/app/layout.tsx (Lines 210–218) & maya-frontend/src/app/+html.tsx (Lines 17–28)

```
// 1) Landing Website (speakwithmaya.com) — maya-landing/src/app/layout.tsx (Lines 210-218)
<head>
  <meta name="facebook-domain-verification" content="4upgmm941w888kmdif1i6kjywtit3o" />
  <Script id="google-tag-manager" strategy="afterInteractive">
    {`...(window,document,'script','dataLayer','GTM-MVNHWL6X');`}
  </Script>
</head>

// 2) Web App (app.speakwithmaya.com) — maya-frontend/src/app/+html.tsx (Lines 17-28)
<meta name="theme-color" content="#0D4EFD" />
<meta name="facebook-domain-verification" content="78qfs00exvb1c95o4ctbms7pq1c0wc" />
<title>SpeakwithMaya - Your Personal AI English Speaking Coach</title>
```

[SCENARIO 2] Automatic Personal Data (PII) Blocker & Privacy Filter

sanitizeDataLayerPayload()

What Happens (Plain English): Every single tracking event passes through a security filter in analytics.ts before reaching window.dataLayer. This filter checks a blocklist of 20 sensitive personal fields (phone, name, email, OTP, card details) and deletes them if present.
Why It Matters for Business / Marketing: Ensures 100% compliance with Sri Lanka PDPA and Meta/Google privacy policies so student phone numbers, names, or payment card details can never be leaked to third-party analytics tools.

Real-Life Example: If a developer passes { event: 'sign_up', method: 'phone_otp', phone_number: '+94771234567' }, the filter automatically strips 'phone_number' and only sends { event: 'sign_up', method: 'phone_otp' }.

Exact Data Sent to GTM: `Strips: phone, phone_number, name, display_name, otp, code, email, card_no, card_masked, cvv, customer_token`

Code File & Line Numbers: maya-frontend/src/utils/analytics.ts (Lines 43–64 & 114–139)

```
// maya-frontend/src/utils/analytics.ts (Lines 43-64 & 114-139)
const FORBIDDEN_PII_KEYS = new Set([
  'phone', 'phone_number', 'name', 'display_name', 'otp', 'code',
  'password', 'email', 'card_no', 'card_masked', 'cvv', 'customer_token',
]);

export function sanitizeDataLayerPayload(payload: Record<string, unknown>) {
  const clean: Record<string, unknown> = {};
  for (const [key, val] of Object.entries(payload)) {
    if (val === undefined || val === null) continue;
    if (FORBIDDEN_PII_KEYS.has(key.toLowerCase())) continue;
    clean[key] = val;
  }
  return clean;
}
```

[SCENARIO 3] New Student Registration via SMS OTP

`sign_up`

What Happens (Plain English): When a student enters their 6-digit SMS code on the OTP screen, the backend tells the app whether this is a brand-new account (res.isNewUser). Only when it is a new registration (!isLoginMode && res.isNewUser) does the app send 'sign_up'.
Why It Matters for Business / Marketing: Separates true new student sign-ups from existing students logging back into their accounts, giving marketing accurate Cost-Per-Registration metrics.

Real-Life Example: Kasun signs up for the first time with his Dialog number and enters the SMS PIN. The app sends 'sign_up' once. Tomorrow, when Kasun logs in again on his laptop, 'sign_up' does NOT fire again.

Exact Data Sent to GTM: `{ event: 'sign_up', method: 'phone_otp' }`

Code File & Line Numbers: maya-frontend/src/utils/analytics.ts (Lines 302–313) & maya-frontend/src/app/onboarding/otp.tsx (Lines 207–212)

```
// 1) Emitter in maya-frontend/src/utils/analytics.ts (Lines 302-313)
export function trackSignUp(options?: { userId?: string; method?: string }): void {
  const dedupKey = options?.userId || 'latest_signup';
  if (options?.userId) {
    const tracked = readTrackedSet(TRACKED_SIGNUPS_STORAGE_KEY, memoryTrackedSignups);
    if (tracked.has(dedupKey)) return;
    addToTrackedSet(TRACKED_SIGNUPS_STORAGE_KEY, memoryTrackedSignups, dedupKey);
  }
  pushDataLayerEvent('sign_up', { method: options?.method || 'phone_otp' });
}

// 2) Call Site in maya-frontend/src/app/onboarding/otp.tsx (Lines 207-212)
const res = await authService.verifyOtp(displayPhone, code, { isLoginMode });
if (!isLoginMode && res.isNewUser) {
  trackSignUp({ userId: res.user?.id, method: 'phone_otp' });
}
```

[SCENARIO 4] Finishing the Onboarding Quiz & Building the Learning Plan

`onboarding_complete`

What Happens (Plain English): After the student answers the onboarding questions (speaking goal, English level, biggest challenge, daily practice minutes), the 'Building Your Plan' screen animates to 100% and fires 'onboarding_complete' once per user ID.
Why It Matters for Business / Marketing: Shows how many registered users complete the onboarding quiz vs. dropping off midway through the questions.

Real-Life Example: Dilini finishes picking 'Career Growth' and '10 mins/day'. As her personalized plan finishes generating at 100%, the app fires 'onboarding_complete' and saves her userId in localStorage so refreshing the page never double-counts it.

Exact Data Sent to GTM: `{ event: 'onboarding_complete' }`

Code File & Line Numbers: analytics.ts (Lines 318–325) | building-plan.tsx (Line 381) | personalized-plan.tsx (Lines 235, 642)

```
// 1) Emitter in maya-frontend/src/utils/analytics.ts (Lines 318-325)
export function trackOnboardingComplete(userId?: string): void {
  const dedupKey = userId || 'current_user';
  const tracked = readTrackedSet(TRACKED_ONBOARDING_STORAGE_KEY, memoryTrackedOnboarding);
  if (tracked.has(dedupKey)) return;
  addToTrackedSet(TRACKED_ONBOARDING_STORAGE_KEY, memoryTrackedOnboarding, dedupKey);
  pushDataLayerEvent('onboarding_complete');
}

// 2) Call Site in maya-frontend/src/app/onboarding/building-plan.tsx (Lines 381-384)
trackOnboardingComplete(OnboardingStore.getState().user?.id);
redirectTimeoutRef.current = setTimeout(() => {
  router.replace('/onboarding/special-offer' as any);
}, 550);
```

[SCENARIO 5] Viewing Pricing, Expanding All Plans & Selecting a Plan Card

`view_plan_selection / view_all_plans / select_plan / continue_starter`

What Happens (Plain English): Tracks how students interact with pricing: (1) 'view_plan_selection' when the pricing screen opens (or when a visitor scrolls to the pricing section on speakwithmaya.com), (2) 'view_all_plans' when they tap 'See more Plans', (3) 'select_plan' with the LKR price when they tap a plan or talk-time pack, and (4) 'continue_starter' if they choose to continue with the Starter plan.
Why It Matters for Business / Marketing: Reveals which plans students are most interested in (e.g., 3-Months Pro at LKR 8,640 vs. Monthly Pro at LKR 5,400 vs. Starter).

Real-Life Example: Amaya opens the Personalized Plan screen ('view_plan_selection'), clicks 'See more Plans' ('view_all_plans'), and taps the Pro card ('select_plan' with plan_name: 'Pro', billing_period: 'monthly', value: 5400, currency: 'LKR').

Exact Data Sent to GTM: `{ event: 'select_plan', plan_name: 'Pro', billing_period: 'monthly' | '3_months', value: 5400, currency: 'LKR' }`

Code File & Line Numbers: analytics.ts (L164–237, 330–370) | personalized-plan.tsx (L236, 643, 716, 819) | upgrade.tsx (L381, 632, 967) | landing/page.tsx (L513, 1546, 1610, 1667)

```
// 1) Selecting a Plan Card in maya-frontend/src/app/onboarding/personalized-plan.tsx (Lines
    715-723)
onPress={() => {
  setSelectedPlanId(plan.id);
  trackSelectPlan(
    resolvePlanAnalyticsDetails({
      planId: plan.id, planName: plan.name,
      billedInfo: plan.billedInfo, priceText: plan.pricePerMonth,
    })
  );
}}

// 2) Choosing to Continue on Starter Plan in personalized-plan.tsx (Lines 642-643)
trackOnboardingComplete(OnboardingStore.getState().user?.id);
trackContinueStarter();
```

[SCENARIO 6] Starting Checkout (PayHere Gateway or 1-Click Saved Card)

`begin_checkout`

What Happens (Plain English): When a student clicks the button to pay for their selected plan or extra talk-time pack, 'begin_checkout' fires right before PayHere opens (or when confirming a 1-click saved card charge). It also saves the plan details and order_id in sessionStorage.
Why It Matters for Business / Marketing: Measures checkout intent — how many users who select a plan actually click to pay — and preserves the order details across browser redirects.

Real-Life Example: Nuwan clicks 'Upgrade to Pro' (LKR 5,400/month). The backend generates order SUB_1730001, and the app fires 'begin_checkout' with value: 5400 and currency: 'LKR' right as the PayHere payment window opens.

Exact Data Sent to GTM: `{ event: 'begin_checkout', plan_name: 'Pro', billing_period: 'monthly', value: 5400, currency: 'LKR' }`

Code File & Line Numbers: analytics.ts (L243–257, 356–365) | personalized-plan.tsx (L537, 579) | upgrade.tsx (L668, 703) | common-popup.tsx (L432, 468)

```
// 1) Emitter in maya-frontend/src/utils/analytics.ts (Lines 356-365)
export function trackBeginCheckout(plan: PlanAnalyticsDetails & { transaction_id?: string
    }): void {
  savePendingCheckoutContext(plan);
  pushDataLayerEvent('begin_checkout', {
    plan_name: plan.plan_name, billing_period: plan.billing_period,
    value: plan.value, currency: 'LKR',
  });
}

// 2) Call Site in maya-frontend/src/app/upgrade.tsx (Lines 696-706)
const params = await paymentApi.getCheckoutParams(planId as any);
const checkoutAnalytics = resolvePlanAnalyticsDetails({
  planId, planName: targetPlan?.name || planName, amount: params.amount,
});
trackBeginCheckout({ ...checkoutAnalytics, transaction_id: params.order_id });
```

[SCENARIO 7] Server-Verified Purchase Conversion (Double Anti-Duplicate Protection)

`purchase`

What Happens (Plain English): The app NEVER fires 'purchase' until the backend server verifies that PayHere actually captured the payment. Once verified, it checks two safety locks: (1) backend firstVerifiedForAnalytics flag on the order record in Supabase, and (2) browser localStorage (maya_tracked_purchases_v1) keyed by transaction_id.
Why It Matters for Business / Marketing: Guarantees 100% accurate revenue and ROAS reporting in Meta Ads and Google Analytics — zero fake purchases from URL manipulation and zero duplicate purchases if a student refreshes the page.

Real-Life Example: Sanduni completes her LKR 8,640 payment for 3-Months Pro. The app calls /v1/payments/orders/:orderId/verify, confirms the payment succeeded on the server, and pushes 'purchase' once with transaction_id and event_id.

Exact Data Sent to GTM: `{ event: 'purchase', transaction_id: 'SUB_...', event_id: 'purchase_SUB_...', plan_name: 'Pro', billing_period: '3_months', value: 8640, currency: 'LKR' }`

Code File & Line Numbers: analytics.ts (L376–398) | upgrade.tsx (L406, 672, 715) | personalized-plan.tsx (L261, 545, 592) | account.tsx (L601) | payhere.service.ts (L1916, 2889)

```
// 1) Emitter in maya-frontend/src/utils/analytics.ts (Lines 376-398)
export function trackVerifiedPurchase(payload: PurchaseAnalyticsPayload): boolean {
  const txId = String(payload.transaction_id || '').trim();
  if (!txId || hasPurchaseBeenTracked(txId)) return false;
  markPurchaseTracked(txId);
  clearPendingCheckoutContext();
  pushDataLayerEvent('purchase', {
    transaction_id: txId, event_id: payload.event_id || `purchase_${txId}`,
    plan_name: payload.plan_name, billing_period: payload.billing_period,
    value: Number(payload.value), currency: 'LKR',
  });
  return true;
}

// 2) Verified Call Site in maya-frontend/src/app/upgrade.tsx (Lines 711-723)
const verifyRes = await paymentApi.verifyOrderWithRetry(txOrderId);
if (verifyRes.verified && verifyRes.transaction && verifyRes.firstVerifiedForAnalytics !==
    false) {
  trackVerifiedPurchase({
    transaction_id: verifyRes.transaction.transactionId,
    event_id: verifyRes.transaction.eventId,
    plan_name: verifyRes.transaction.planName,
    billing_period: verifyRes.transaction.billingPeriod,
    value: verifyRes.transaction.value,
    currency: 'LKR',
  });
}
```

[SCENARIO 8] Payment Declined vs. User Closing the Payment Window

`payment_failed / payment_cancelled`

What Happens (Plain English): If a payment does not complete, the app distinguishes between two cases: (1) 'payment_failed' when the card is declined or PayHere returns an error, and (2) 'payment_cancelled' when the student voluntarily closes/dismisses the PayHere checkout popup.
Why It Matters for Business / Marketing: Helps the team see whether checkout drop-offs are caused by bank/card declines (payment_failed) or user hesitation (payment_cancelled).

Real-Life Example: If a user closes the PayHere modal without entering a card, 'payment_cancelled' fires. If their card is declined by the bank, 'payment_failed' fires with the plan name, value, and transaction_id.

Exact Data Sent to GTM: `{ event: 'payment_failed' | 'payment_cancelled', plan_name: 'Pro', billing_period: 'monthly', value: 5400, currency: 'LKR', transaction_id: '...' }`

Code File & Line Numbers: analytics.ts (Lines 403–437) | upgrade.tsx (L421, 444, 686, 728, 738) | personalized-plan.tsx (L276, 299, 561, 607, 617) | account.tsx (L614, 637)

```
// maya-frontend/src/app/upgrade.tsx (Lines 737-746)
} else if (result.status === 'dismissed') {
  trackPaymentCancelled({
    ...checkoutAnalytics,
    transaction_id: params.order_id,
  });
} else if (result.status === 'error') {
  trackPaymentFailed({
    ...checkoutAnalytics,
    transaction_id: params.order_id,
  });
}
```

[SCENARIO 9] AI Speaking Practice Call Start & Completion `session_start / session_complete`

What Happens (Plain English): When a student starts a live voice call with Maya and the Gemini Live connection opens, 'session_start' fires once for that session ID (auto-reconnects during the same call are ignored). When the student finishes the call and views their feedback report, 'session_complete' fires with the exact duration in seconds.
Why It Matters for Business / Marketing: Tracks student engagement, product activation (first voice call after sign-up), and average speaking duration per session.

Real-Life Example: Chathura starts a speaking practice session ('session_start'). After practicing for 5 minutes (300 seconds), the call completes and sends { event: 'session_complete', session_duration: 300 }.

Exact Data Sent to GTM: `{ event: 'session_start' } --> { event: 'session_complete', session_duration: 300 }`

Code File & Line Numbers: maya-frontend/src/utils/analytics.ts (Lines 442–460) & maya-frontend/src/hooks/useLiveCall.ts (Lines 653–657, 1488–1491)

```
// 1) Emitters in maya-frontend/src/utils/analytics.ts (Lines 442-460)
export function trackSessionStart(sessionId?: string): void {
  if (sessionId) {
    if (memoryTrackedSessionsStarted.has(sessionId)) return;
    memoryTrackedSessionsStarted.add(sessionId);
  }
  pushDataLayerEvent('session_start');
}

export function trackSessionComplete(durationSeconds: number, sessionId?: string): void {
  if (sessionId) {
    if (memoryTrackedSessionsCompleted.has(sessionId)) return;
    memoryTrackedSessionsCompleted.add(sessionId);
  }
  const safeDuration = Math.max(0, Math.round(Number(durationSeconds) || 0));
  pushDataLayerEvent('session_complete', { session_duration: safeDuration });
}

// 2) Call Sites in maya-frontend/src/hooks/useLiveCall.ts (Lines 653-657 & 1488-1491)
onOpen: () => {
  setIsConnected(true);
  trackSessionStart(sessionIdRef.current);
}
// ... when session completes:
if (shouldNavigate) {
  trackSessionComplete(secs, sessionIdRef.current);
}
```

3. How Non-Technical Team Members Can Verify Tracking (3 Quick Checks)

What You Want to Check	Simple Steps (No Coding Required)	What You Should See
1. Facebook Domain Verification (Landing & App)	Open **speakwithmaya.com** or **app.speakwithmaya.com** in Chrome, right-click anywhere, choose **View Page Source**, and press **Ctrl+F** to search for `facebook-domain-verification`.	Inside <head>: • Landing: `4upgmm941w888kmdif1i6kjywtit3o` • App: `78qfs00exvb1c95o4ctbms7pq1c0wc`
2. Google Tag Manager Live Events (Tag Assistant)	Open Google Tag Manager → click **Preview** → enter **https://app.speakwithmaya.com**. Click through plans or complete a step.	In the left sidebar of Tag Assistant, events appear in real time: `view_plan_selection`, `select_plan`, `begin_checkout`, `purchase`.
3. Privacy & No Personal Data Leak	In GTM Preview mode, click on any fired event (such as `sign_up` or `purchase`) and open the **Data Layer** tab.	Only safe business values (`plan_name`, `value`, `currency: 'LKR'`, `transaction_id`) appear — never phone numbers, names, or OTPs.